

Problem Statement 1 : ECG Signal Processing

1. Introduction

The objective of this project is to design and simulate a signal processing system for **ECG (Electrocardiogram)** signals, which are vital for monitoring heart health in biomedical applications. The project consists of two main tasks: simulating a realistic ECG signal and designing an analog circuit for amplifying and filtering the signal. This will provide students with practical experience in signal processing, circuit design, and simulation using LTSpice.

2. Objective

The goal is to simulate a realistic ECG waveform, process it through an analog circuit, and enhance the signal for real-time analysis. The system will simulate ECG signal generation, introduce noise, and process the signal through an amplification and filtering circuit. After processing the signal, the final signal must be displayed in **LTSpice**.

The tasks are broken down as follows:

- **Task 1:** Generate and simulate a realistic ECG waveform, introduce noise, and process it for signal analysis.
- **Task 2:** Design an analog circuit to amplify and filter the ECG signal, removing noise while maintaining the integrity of the original waveform, and display the final processed signal in **LTSpice**.

3. Background

An **ECG signal** is a graphical representation of the electrical activity of the heart, consisting of the PQRST components: P wave, QRS complex, and T wave. In real-world applications, ECG signals are often corrupted by noise (e.g., baseline wander, 50/60 Hz interference), which can distort the accuracy of diagnosis if not filtered out.

The primary objectives of this project are to:

1. Simulate the ECG signal accurately in terms of shape, amplitude, and timing.
2. Design an analog circuit to process the ECG signal by amplifying the weak signal and filtering out unwanted noise.
3. Display the final processed signal after filtering in **LTSpice**.

4. System Design Requirements

4.1 Task 1: ECG Signal Simulation

1. ECG Waveform Generation:

- Use **LTSpice** to simulate a synthetic ECG signal.
- The ECG waveform should accurately represent the **PQRST** components (P wave, QRS complex, and T wave).
- Ensure the signal has the appropriate amplitude (typically in millivolt range, around 1 mV).
- Set the **timing characteristics** to match a typical ECG signal (e.g., heart rate of 60-100 bpm, typical duration of PQRST cycle).

2. Noise Introduction:

- **50/60 Hz Interference**: Simulate power line interference that can affect the ECG signal.
- **Baseline Wander**: Introduce low-frequency fluctuations in the ECG signal, typically caused by body movements, respiration, etc.

3. LTSpice Setup:

- Simulate the ECG signal waveform and introduce the noise components for testing.
- The simulation should show the effects of noise on the clean ECG signal to be used in further signal processing.

4.2 Task 2: Analog Signal Processing Circuit Design

1. Amplification:

- Design a **multi-stage amplification circuit** using operational amplifiers to increase the amplitude of the weak ECG signal.
- The circuit should have a **gain** that amplifies the ECG signal from its typical millivolt range to a measurable level (e.g., 1-5 V).

2. Filtering:

- **High-pass Filter**: Design a **high-pass filter** to remove low-frequency components, specifically to eliminate **baseline wander** (below 0.5 Hz).
- **Low-pass Filter**: Design a **low-pass filter** to eliminate high-frequency noise, especially **50/60 Hz interference** and other higher-frequency noise components (above 50 Hz).
- The filtering circuit should preserve the integrity of the ECG signal while removing unwanted noise.

3. Circuit Design in LTSpice:

- Use **LTSpice** to simulate and optimize the circuit design.
- Ensure the circuit meets the frequency and amplitude specifications for proper ECG signal processing.

4. Display the Final Processed Signal:

- After implementing the signal amplification and filtering circuit, **display the final processed signal** in LTSpice.
- The final waveform should show the ECG signal after noise has been filtered out and the signal has been appropriately amplified.
- This allows for the verification of the signal's quality and correctness post-processing.

5. Functional Requirements

- **ECG Simulation:** The synthetic ECG signal should closely match the real-world ECG signal in terms of amplitude, waveform shape, and timing characteristics.
- **Signal Processing:** The processing circuit should be able to:
 1. **Amplify** the ECG signal to a measurable level.
 2. **Filter** noise components, including baseline wander and 50/60 Hz interference, to restore the clarity of the ECG waveform.
- **LTSpice Simulation:** Both the ECG signal generation and the analog processing circuit must be simulated and tested in **LTSpice**, with the final processed signal displayed.

6. Tools and Software

- **LTSpice:** For simulating the ECG waveform and analog signal processing circuits.

7. Signal Processing Design Details

7.1 Amplification Circuit

- The amplification stage uses operational amplifiers configured in **non-inverting** or **inverting amplifiers** to provide the necessary gain.
- Gain is calculated as $G = 1 + \frac{R_f}{R_{in}}$ for non-inverting and $G = -\frac{R_f}{R_{in}}$ for inverting, where R_f is the feedback resistor and R_{in} is the input resistor.
- The gain should be sufficient to increase the ECG signal to a measurable level for further processing.

7.2 High-pass Filter Design

- A **high-pass filter** will be used to eliminate low-frequency baseline wander (below 0.5 Hz).
- A simple **RC filter** can be designed using a resistor and capacitor in series with the output taken after the resistor.
- The cutoff frequency f_c is determined by the formula $f_c = \frac{1}{2\pi RC}$.

7.3 Low-pass Filter Design

- A **low-pass filter** will be used to remove high-frequency noise, particularly the 50/60 Hz interference.
- The cutoff frequency should be around 50 Hz to block this noise.
- An **RC filter** can again be used, with a similar design approach as the high-pass filter.

8. Expected Results

- After simulation in LTSpice, the final signal should have:

1. **A clean ECG waveform** with properly amplified amplitude.
 2. **No baseline wander** and minimal low-frequency noise.
 3. **No 50/60 Hz interference** and reduced high-frequency noise.
- The processed signal should be displayed in **LTSpice** to visualize the effects of the amplification and filtering.

9. Troubleshooting and Considerations

- **Noise Level Adjustment:** Ensure the level of noise introduced during the simulation is realistic and not overwhelming.
- **Filter Tuning:** Carefully adjust the cutoff frequencies of the filters to ensure that relevant ECG components are not filtered out.
- **Simulation Accuracy:** Ensure LTSpice simulations match expected results and accurately simulate real-world conditions like noise and interference.

10. Conclusion

This project will provide students with hands-on experience in designing signal processing circuits for **ECG signals**, a critical area in **biomedical engineering**. By simulating both the signal generation and processing stages in LTSpice and displaying the final processed signal, students will learn how to amplify and filter ECG signals, making them suitable for real-time analysis and medical applications.

Problem Statement 2 : Closed-Loop Control on a Boost Converter using PID

1. Introduction

This problem statement involves designing a closed-loop control system for a boost converter to output a specific voltage by varying the duty cycle using a **PID controller**. The simulation will be done in **MATLAB Simulink**. The next task is to implement an **Arduino-based controller** that adjusts the PWM duty cycle, which is input to a MOSFET to control the output voltage of the boost converter.

2. Objective

The primary objective is to design a system that simulates the closed-loop control of a boost converter. The system should use a PID controller to adjust the duty cycle based on the error between the desired output voltage and the actual output voltage.

1. **Simulink Simulation:** Create a Simulink model of the boost converter with a closed-loop PID controller to adjust the duty cycle based on feedback.
2. **Arduino Implementation:** Write Arduino code that implements the PID control logic to adjust the PWM signal's duty cycle, which controls the MOSFET for the boost converter.

3. Background

A **boost converter** is a type of DC-DC converter that steps up the input voltage to a higher output voltage. The operation of the boost converter is regulated by adjusting the duty cycle of the pulse-width modulation (PWM) signal that controls the MOSFET. Frequency of PWM should be high (8KHz to 80KHz) be high for an efficient boost converter.

A **PID controller** (Proportional-Integral-Derivative controller) is used in feedback systems to minimize the error between the desired setpoint (desired output voltage) and the actual system output. The PID controller computes an output that adjusts the duty cycle of the PWM signal to drive the error towards zero.

4. System Design Requirements

4.1 MATLAB Simulink Model

- **Boost Converter Circuit:** Use Simulink's electrical and power systems library to design a boost converter model.
- **PID Controller:** The controller should compute the duty cycle based on the difference between the desired output and the actual output voltage.
- **Voltage Feedback:** A voltage sensor will be simulated to measure the output voltage of the boost converter.

4.2 Arduino Code

The Arduino code should:

- **Read the actual output voltage** (simulated through ADC, e.g., using `analogRead`).
- **Compute the error** between the desired and actual output.
- **Apply the PID formula** to calculate the new duty cycle.
- **Output PWM signal** to control the MOSFET gate to adjust the output voltage.

5. Functional Requirements

- The system should output a desired voltage (e.g., 12V) by adjusting the duty cycle.
- The PID controller should be able to dynamically adjust the duty cycle to reach and maintain the desired voltage.
- The Arduino code should be able to generate a PWM signal with a duty cycle controlled by the PID controller.

6. Tools and Software

- **MATLAB Simulink**: For simulating the closed-loop boost converter system and PID controller.
- **Arduino IDE**: For writing and uploading the Arduino code to the microcontroller.

7. PID Control Algorithm

7.1 PID Control Theory

The PID controller works by calculating three components based on the error:

- **Proportional**: The error term that scales the controller output directly to the current error.
- **Integral**: The sum of all past errors to eliminate steady-state error.
- **Derivative**: The rate of change of the error to predict future error trends and stabilize the system.

The formula for the PID controller is:

$$\text{Output} = K_p \times \text{Error} + K_i \times \text{Integral of Error} + K_d \times \text{Derivative of Error}$$

Where:

- K_p , K_i , and K_d are the PID constants (proportional, integral, and derivative gains).

7.2 Arduino Code Implementation

- **Input**: The actual voltage measured (analog input).
- **Output**: PWM duty cycle, which is sent to the MOSFET gate.

- The Arduino will implement the following key steps:
 1. **Read the actual voltage.**
 2. **Calculate the error** between the setpoint and actual voltage.
 3. **Compute PID:** Use the PID algorithm to determine the duty cycle.
 4. **Generate PWM signal** using `analogWrite()`.

8. Simulation Steps in Simulink

1. **Create the Boost Converter Model:** Use Simulink blocks to create a boost converter, including the switch (MOSFET), inductor, capacitor, and load resistor.
2. **Add the PID Controller:** Use the PID Controller block from Simulink and connect it to the PWM generation block.
3. **Feedback Loop:** Use the voltage measurement block to provide feedback to the PID controller.
4. **Simulate the System:** Run the simulation and observe the output voltage's response to changes in the PID controller settings.

9. Expected Results

- The boost converter should output the desired voltage (e.g., 12V) based on the PID-controlled duty cycle.
- The system should quickly adjust to changes in the desired voltage and maintain stable operation.

10. Troubleshooting and Considerations

- **Tuning PID Parameters:** The PID constants K_p , K_i , and K_d may need to be tuned to achieve the desired system response.
- **PWM Resolution:** Ensure that the Arduino's PWM resolution is sufficient to provide the necessary precision in duty cycle control.
- **System Stability:** Proper tuning of the PID controller is crucial for system stability and achieving steady-state operation without oscillations or overshooting.

11. Conclusion

This project will help students understand the implementation of closed-loop control systems in power electronics. It combines theoretical knowledge of PID control with practical application in power electronics systems like boost converters. The Simulink simulation and Arduino code work together to provide a hands-on learning experience in embedded control systems.